```python
from flask import Flask, render_template, request, redirect, session
from detector import start_monitoring, stop_monitoring
from database import init_db, get_alerts, register_user, login_user

# ONLY ADD (watchdog integration)
from watchdog_detector import start_watchdog, stop_watchdog

app = Flask(__name__)
app.secret_key = "secret123"

init_db()


@app.route("/")
def home():
    if "user" not in session:
        return redirect("/login")

    db_alerts = get_alerts()
    alerts_list = [row[1] for row in db_alerts]

    return render_template("index.html", alerts=alerts_list, user=session["user"])


# LOGIN
@app.route("/login", methods=["GET", "POST"])
```

```python
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]

        user = login_user(username, password)

        if user:
            session["user"] = username
            return redirect("/")
        else:
            return "Invalid login"

    return render_template("login.html")


# REGISTER
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]

        success = register_user(username, password)

        if success:
```

```
            return redirect("/login")
        else:
            return "User already exists"

    return render_template("register.html")


@app.route("/logout")
def logout():
    session.pop("user", None)
    return redirect("/login")


# OLD DETECTOR (UNCHANGED)
@app.route("/start")
def start():
    start_monitoring()
    return redirect("/")


@app.route("/stop")
def stop():
    stop_monitoring()
    return redirect("/")


# □ WATCHDOG (ONLY ADDITION)
@app.route("/watchdog-start")
def watchdog_start():
    start_watchdog()
    return redirect("/")

```

```python

@app.route("/watchdog-stop")
def watchdog_stop():
    stop_watchdog()
    return redirect("/")


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=10000)
```

```
web: python app.py
```

```
adhi123,123456

```

```python
import os

folder_path = "test_folder"

for file in os.listdir(folder_path):
    if file.endswith(".locked"):
        old_path = os.path.join(folder_path, file)
        new_name = file.replace(".locked", "")
        new_path = os.path.join(folder_path, new_name)

        os.rename(old_path, new_path)
        print("Restored:", new_name)
```

```python
import sqlite3
import hashlib


def init_db():
    conn = sqlite3.connect("alerts.db")
    cursor = conn.cursor()

    # Alerts table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS alerts (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        message TEXT,
        timestamp TEXT
    )
    """)

    # Users table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE,
        password TEXT
    )
    """)

    conn.commit()
    conn.close()


# ==========================
```

```python
# 🔐 PASSWORD HASHING
# ==========================

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()


# ==========================
# 🚨 ALERT FUNCTIONS
# ==========================

def insert_alert(message, timestamp):
    conn = sqlite3.connect("alerts.db")
    cursor = conn.cursor()

    cursor.execute(
        "INSERT INTO alerts (message, timestamp) VALUES (?, ?)",
        (message, timestamp)
    )

    conn.commit()
    conn.close()


def get_alerts():
    conn = sqlite3.connect("alerts.db")
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM alerts ORDER BY id DESC")
    data = cursor.fetchall()
```

```python

    conn.close()
    return data


# ==========================
# 🔐 USER FUNCTIONS (UPDATED)
# ==========================

def register_user(username, password):
    conn = sqlite3.connect("alerts.db")
    cursor = conn.cursor()

    try:
        hashed_pw = hash_password(password)

        cursor.execute(
            "INSERT INTO users (username, password) VALUES (?, ?)",
            (username, hashed_pw)
        )

        conn.commit()
        return True

    except:
        return False

    finally:
        conn.close()


```

```python
def login_user(username, password):
    conn = sqlite3.connect("alerts.db")
    cursor = conn.cursor()

    hashed_pw = hash_password(password)

    cursor.execute(
        "SELECT * FROM users WHERE username=? AND password=?",
        (username, hashed_pw)
    )

    user = cursor.fetchone()
    conn.close()

    return user
```

```python
import os
import time
import threading
import datetime
from database import insert_alert

folder_path = "test_folder"

alerts = []
running = False


def add_alert(message):
    print(message)
    alerts.append(message)

    timestamp = str(datetime.datetime.now())
    insert_alert(message, timestamp)


def monitor_folder():
    global running

    previous_locked = set()

    while running:
        try:
            current_files = os.listdir(folder_path)

            current_locked = set(
                [f for f in current_files
```

```
if f.endswith(".locked")]
            )

            new_locked = current_locked - previous_locked

            for file in new_locked:
                add_alert(f"{datetime.datetime.now()} ⚠ Detected: {file}")

            previous_locked = current_locked

        except Exception as e:
            print("Error:", e)

        time.sleep(1)


def start_monitoring():
    global running
    if not running:
        running = True
        thread = threading.Thread(target=monitor_folder)
        thread.daemon = True
        thread.start()


def stop_monitoring():
    global running
    running = False
```

```python
import os
import time

folder_path = "test_folder"

files = [f for f in os.listdir(folder_path) if f.endswith(".txt")]

print("Files found:", files)

for file in files:
    old_path = os.path.join(folder_path, file)
    new_path = os.path.join(folder_path, file.replace(".txt", ".locked"))

    if os.path.exists(old_path):
        os.rename(old_path, new_path)
        print("Encrypted:", file)
        time.sleep(2)
```

```
Flask
watchdog
```

```python
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import time
import datetime
from database import insert_alert

folder_path = "test_folder"


class RansomwareHandler(FileSystemEventHandler):

    def on_created(self, event):
        if not event.is_directory:
            message = f"{datetime.datetime.now()} 🆕 CREATED: {event.src_path}"
            print(message)
            insert_alert(message, str(datetime.datetime.now()))

    def on_modified(self, event):
        if not event.is_directory:
            message = f"{datetime.datetime.now()} ⚠️ MODIFIED: {event.src_path}"
            print(message)
            insert_alert(message, str(datetime.datetime.now()))

    def on_deleted(self, event):
        if not event.is_directory:
            message = f"{datetime.datetime.now()} 🗑 DELETED: {event.src_path}"
```

```
            print(message)
            insert_alert(message, str(datetime.datetime.now()))


observer = Observer()
observer.schedule(RansomwareHandler(), folder_path, recursive=False)


def start_watchdog():
    observer.start()
    print("Watchdog started...")


def stop_watchdog():
    observer.stop()
    observer.join()
    print("Watchdog stopped")
```